

Heap Overflows and Buffer Overflow Defenses

CS 465 · Introduction to Security and Privacy · Fall 2026

BYU



Heap Overflows

buffer overflow countermeasures and why they are hard to deploy



Beyond the Stack

The heap can also be overflowed

Buffers also live in the heap and the static area. Overflow one and you corrupt whatever sits just above it.

Writable, executable

The heap and BSS were long left writable, which is necessary, and executable, which is not. (Modern systems don't make it executable.)

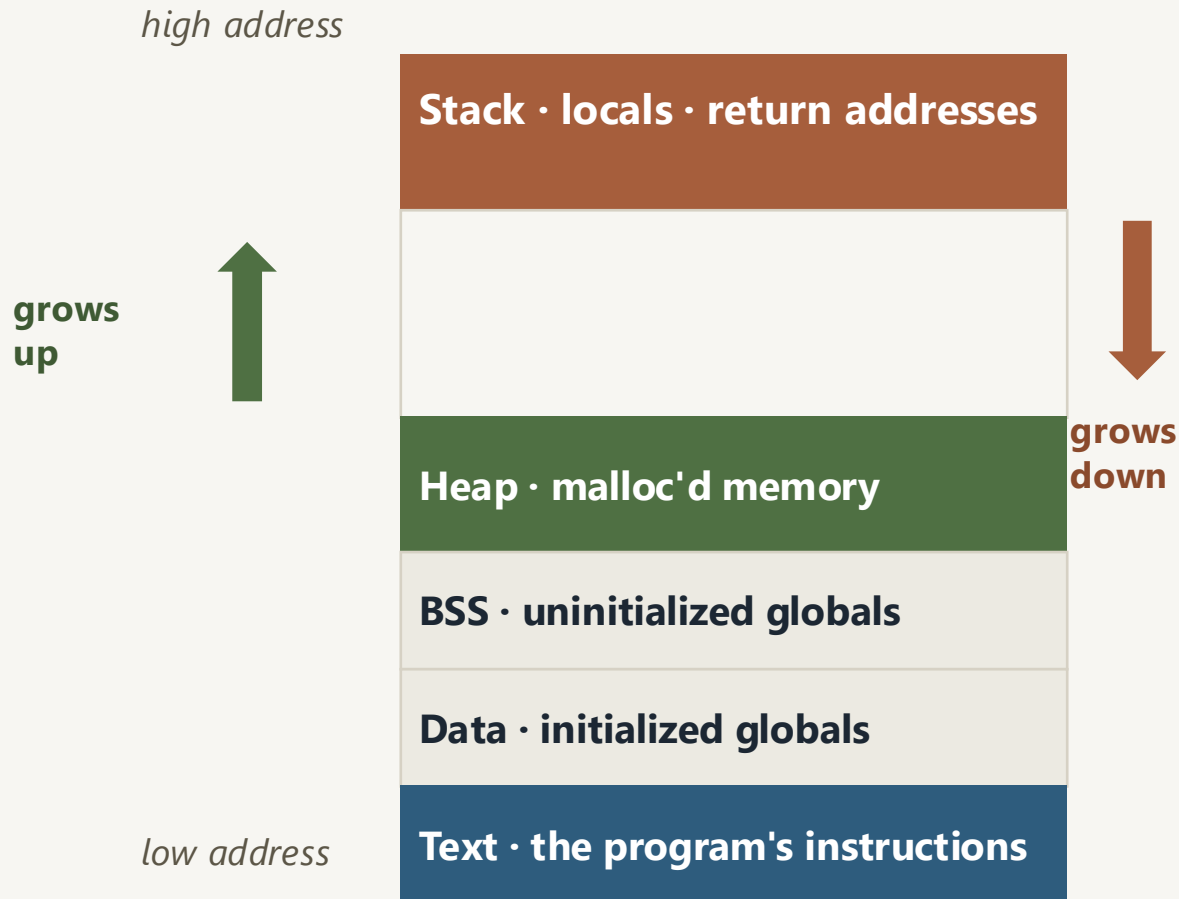
A different entry point

No return address here. The prizes are permission flags, function pointers and allocator metadata.

The layout is variable

Dynamic allocation varies across systems and is far less predictable than the stack, so attackers experiment first to build a roadmap.

Where the Heap Sits



Why the heap

Heap and stack grow toward each other through the free space between them.

Heap, BSS and Data are all writable — and were long left executable too. Any writable segment is fair game.

Writable and Executable

A heap that is writable and executable lets attacker data become attacker code.

Find the buffer

An exploitable heap buffer, plus a useful variable at a nearby higher address.

Do not crash

Everything in between gets clobbered. If that kills the program, it is a denial of service, not an exploit.

Overflowing into a Permissions Variable

(based on Figure 6.6a)

before · addresses increase to the right

meta-data	ABCD	0123	89AB	CDEF	permsY = 0
-----------	------	------	------	------	------------

after · a copy of five words into a four-word buffer

meta-data	AAAA	AAAA	AAAA	AAAA	permsY = 1
-----------	------	------	------	------	------------

↑ *the four words bufferX owns*

↑ *overwritten — FALSE becomes TRUE*

The overflow only has to reach permsY

bufferX owns four words. A fifth write lands in whatever the allocator put next, and the program's next permission check reads TRUE. No code was injected, and nothing crashed.

Demo 1 — Changing a User's Role

```
# github.com/BYU-CS465/software-security  
$ cd heap-overflow  
$ ./heap
```

What the program does

- Allocates a buffer and a role on the heap, then copies input into the buffer without checking its length.
- The role sits at a higher address than the buffer — Figure 6.6a, running.

What to watch

- Short input: the role stays what it was.
- Longer input: the copy runs past the buffer and lands in the role field.
- This exploit is trivial — no offsets to hunt, just keep typing.

Overflowing into a Function Pointer

(based on Figure 6.6b)

before · addresses increase to the right

meta-data	ABCD	0123	89AB	CDEF	fnptrZ → fn1
-----------	------	------	------	------	--------------

after · a copy of five words into a four-word buffer

meta-data	AAAA	AAAA	AAAA	AAAA	fnptrZ → heap
-----------	------	------	------	------	---------------

↑ *the four words bufferX owns*

↑ *now points at attacker bytes*

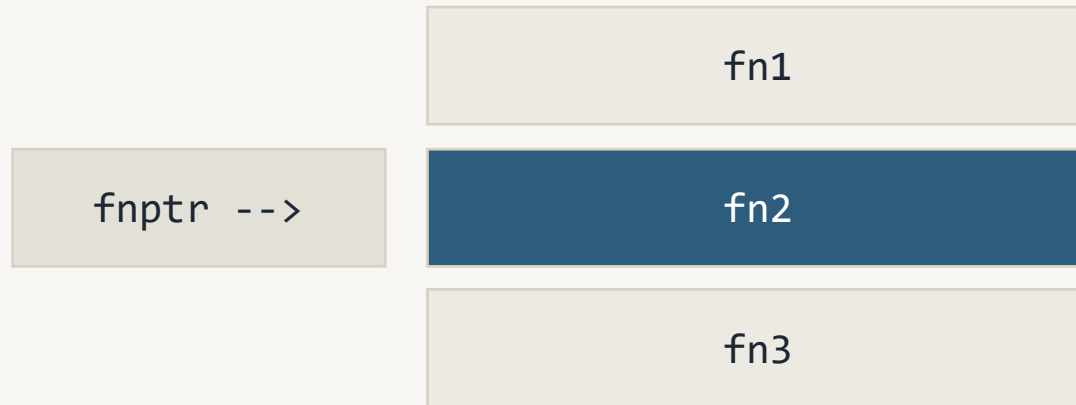
Same overflow, but this time a function pointer

fnptrZ holds the address of a function the program is about to call. Overwrite it and the next call goes wherever you wrote. That is the same control transfer a stack attack gets from a return address, and the heap version is often easier to reach.

Corrupting a Function Pointer

(based on Figure 6.7)

(a) before manipulation



the pointer holds the address of fn2

(b) after manipulation



the same call now lands in attacker code

The call site never changes. Getting the code there is a separate problem.

Demo 2 — Overwriting a Function Pointer

```
$ gdb ./heap2                # gef
gef> break after_copy
gef> run < $(python3 exploit.py --pattern 200)
gef> print fnptr              # what landed there?
gef> pattern search 0x6161...  # offset
$ python3 exploit.py --offset N --target 0x... | ./heap2
```

Finding the offset

1. Break after the copy, overflow with a cyclic pattern, print the pointer.
2. Feed that value to pattern search — same technique as the stack exploit.
3. Pad to the offset, then write the address you want called.

What Gets Corrupted — Heap or Stack

- a) Stack-based pointers, including return addresses and frame pointers. • **stack**
- b) Function pointers in the stack, heap or static area — including any lookup table: jump table, dispatch table, vtable. • **heap, Fig 6.6b**
- c) Addresses used by setjmp and longjmp. • **either**
- d) Indirectly, data used in a branching test — the branch goes to a fixed address, but the wrong one. • **heap, Fig 6.6a**

Today's exploits use (b) and (d); (a) is what we did last time on the stack.

Jump Tables and Saved Contexts

```
MyJump(int c) {  
    switch (state) {  
        case 0: goto func0label;  
        case 1: goto func1label;  
        case 2: goto func2label;  
    }  
}
```

A table of destinations

Dispatch tables and vtables work the same way.

setjmp and longjmp

Corrupt it and longjmp resumes anywhere.

Building a Heap Spray

```
var sled = nops(0x1000);  
var block = sled + shellcode;  
var spray = new Array();  
for (i = 0; i < 10000; i++) {  
    spray[i] = block;  
}
```

```
one sprayed block  
[ NOP NOP NOP ... NOP ]  
[ shellcode           ]  
x 10,000 blocks  
= 1-100 MB of heap
```

Where the blocks come from

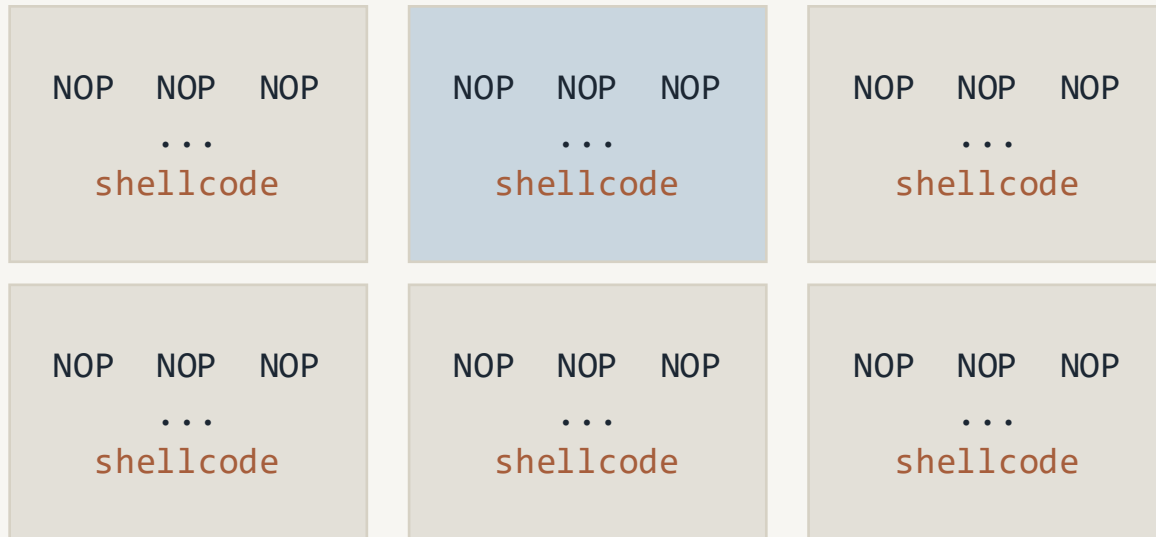
- A script in a web page — the classic drive-by download.
- 10,000 strings in a plain JavaScript loop.
- A loaded image can trigger the same allocations.

Why the Spray Works

(based on Figure 6.8)

`fnptr corrupted --> some address in the heap`

heap sprayed with thousands of identical blocks



the pointer lands in a sled and slides forward into the shellcode

Any address in the spray will do

A stack exploit needs one exact address. Here the attacker only has to land somewhere in the sprayed region: the pointer hits a sled and execution slides forward until it reaches the shellcode.

This is step 1 only. Some other bug has to corrupt the pointer.

Why Heap Spraying Is Hard to Stop

The spray never breaks a rule: every allocation is legal, and the script is ordinary JavaScript.

Nothing to catch

No overflow, no type violation. A page asking for 10,000 strings looks like a page asking for 10,000 strings.

Randomizing doesn't work

Layout randomization assumes the attacker has to guess one address. When most of the heap is sled, nearly any guess works.

Generic Exploit Steps

Stack or heap, most buffer overflow exploits come down to the same three functional steps.

1 • Inject or locate code

Get the desired code into the target's address space. If existing code already does the job, no injection is needed — only its address.

2 • Corrupt control-flow data

Overwrite a structure that decides where execution goes: a return address, a function pointer, a table entry. Often the same overflow.

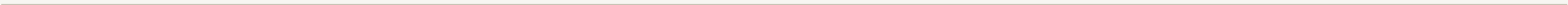
3 • Seize control

Transfer to the step 1 code. Sometimes this means doing nothing at all — just wait for the program to use what you corrupted.

Heap spraying is step 1, industrialised. It floods the address space with copies of the code and leaves step 2 to a separate bug.

Buffer Overflow Defenses

buffer overflow countermeasures and why they are hard to deploy



Defending Against Overflows

- 1) Non-executable stack and heap — DEP, NX bit • **run-time**
- 2) Stack and heap canaries, shadow stacks, pointer encoding • **run-time**
- 3) Run-time bounds checking, instrumented by the compiler • **both**
- 4) Memory layout randomization — ASLR, PIE, secure heap allocators • **run-time**
- 5) Type-safe languages, or safe dialects of C • **compile-time**
- 6) Safe C libraries in place of the unchecked libc string functions • **compile-time**
- 7) Static analysis tools on source and binaries • **compile-time**

Compile-time costs developer effort; run-time costs performance. Attackers have found ways around several of these.

Non-Executable Stack and Heap

Data Execution Prevention (DEP) marks the stack, heap and BSS as data only. Hardware enforces it with the **NX (no-execute) bit**: those pages can be read and written, but the instruction pointer may never fetch from them.

```
$ gcc heap2.c -o heap2           # NX on by default
$ gcc -z execstack heap2.c       # turns it off
$ readelf -l heap2               # look at STACK section -- RW, no E
```

May not be switched on

Old software needs an executable stack, an attacker turns it off, or a JIT must run code it just wrote.

It blocks running, not writing

Corrupting a variable needs no execution at all. Catching the write itself is what canaries do.

Stack Canaries

a random word is placed between the locals and the return address

buffer[0..3]	locals	locals	canary	saved FP	return address
--------------	--------	--------	--------	----------	----------------

an overflow reaching the return address must cross the canary first

AAAA	AAAA	AAAA	AAAA	AAAA	0xdeadbeef
------	------	------	------	------	------------

↑ *canary destroyed*

↑ *epilogue checks this*

The function checks its own canary before it returns

The compiler emits both halves: the prologue stores the canary, the epilogue compares it and calls `__stack_chk_fail` to abort if it changed. On by default in gcc, off with `-fno-stack-protector`.

Address Space Layout Randomization (ASLR) and Position-Independent Executable (PIE)

ASLR — The kernel gives the stack, heap and libraries a new base address on every run.

PIE — Compiler builds code to run at any base, so ASLR can move the code and data.

first run

0x7ffd_a41c	stack
0x7f2b_0e90	libc
0x5566_12a0	heap
0x5566_1000	text

the attacker's hard-coded address works

second run, same binary

0x7ffc_38b0	stack
0x7f91_cd10	libc
0x5601_ff40	heap
0x5601_f000	text

the same address now points at nothing useful

Note: ASLR is only as strong as its implementation — several real deployments randomize far less than advertised.

Secure Heap Allocator

What it does

Randomize where blocks land

Each allocation goes to an unpredictable address rather than neatly after the last one.

Guards its own bookkeeping

Block size and free-list links move to a separate region, carry a checksum, and are validated before use.

Why it matters

Neighbors stop being predictable

The attacker cannot count on a useful variable sitting just above the buffer they can overflow.

malloc cannot be turned on you

Corrupting those fields would let an attacker make the allocator write to an address of their choosing.

Writing Safer Code

Type-safe languages

Rust, Java, C# enforce bounds and control casting. Safe dialects of C are the halfway option.

Safe C libraries

Replace libc string functions like strcpy, which copy without checking any bound.

Bounds checking

The compiler instruments the code and a run-time checks every access. You pay for it in performance.

Static analysis tells you which lines to fix

The flagged code gets rewritten — but that is a handful of fixes, not the whole program in a new language. Tools miss some bugs and raise false alarms.

Adoption Barriers

We know how to stop most of these attacks. Deploying the fixes worldwide is the hard part.

No governing body

No company, country or standards group can impose a rule on all software everywhere — even if the ideal fix were known.

Backwards compatibility

Anything that breaks interoperability, or stops a program people still use, meets opposition immediately.

Incomplete solutions

A fix covering some vulnerabilities at real deployment and performance cost loses the cost-benefit argument.

And the legacy base is enormous. Rewriting the world's C, C++ and assembler is blocked by cost, scarce expertise, and the risk of disrupting critical systems.

Coda: Attackers Adapt

Every defense raises the cost without ending the game.

Return-to-libc is non-required reading.

Advanced reading: ASLR timing side channels, return-oriented programming, position-independent code reuse

Return-to-libc: No Injected Code at All

Point the return address at code the system already trusts — `system()` in `libc` — with arguments the attacker laid out on the stack.

How it avoids defenses

Non-executable memory stops code running from the stack or heap. It does nothing about a jump into `libc`, because `libc` is supposed to be executable.

How far it goes

`system("/bin/sh")` is one call away from a shell. Chaining fragments of existing code this way is return-oriented programming (§6.5, advanced reading).